
Développement d'applications distribuées

CER | JWT Docker

Lundi 8 juin 2026

TABLE DES MATIÈRES

AAV	2
Étude de cas : JWT Docker	3
Mots clés	3
Contexte	3
Problématique	3
Contraintes	3
Généralisation	4
Livrables	4
Pistes de solutions	4
Plan d'actions	4
Définition : Mots clés	5
Prototype MERN	5
Docker (conteneurisation)	5
JWT (JSON Web Token)	6
Jetons / Tokens	6
Authentification	7
API Gateway	7
Micro-services	8
Multi-conteneurs	8
Autorisation (Contrôle d'accès)	9
Analyse et résolution : JWT Docker	9
1. Mécanisme d'authentification (JWT)	9
2. Architecture cible	10
3. Génération et validation des JWT	10
4. Contrôle d'accès back-end	12
5. Contrôle d'accès front-end	12
6. Conteneurisation et déploiement	14
7. Architecture retenue	14
Conclusion	16
Capitalisation	17
Ressources	18

- 1 Expliquer le fonctionnement d'un service tiers d'authentification
- 5 Gérer la sécurité de l'application à l'aide de jetons
- 5 Mettre en place le contrôle d'accès dans une application backend
- 4 Proposer un contrôle d'accès pour une application front
- 3 Réaliser un système d'authentification dans une API

ÉTUDE DE CAS : JWT DOCKER

MOTS CLÉS

- Prototype MERN
- Docker
- JWT
- Jetons / Tokens
- Authentification
- API Gateway
- Micro-services
- Multi-conteneurs
- Autorisation (Contrôle d'accès)

CONTEXTE

Le développeur Yanis doit sécuriser son application web MERN en micro-services, via une API Gateway et automatiser le déploiement de ses services.

PROBLÉMATIQUE

Comment sécuriser l'accès aux micro-services d'une application web MERN, en utilisant des jetons JWT pour l'authentification et le contrôle d'accès, tout en automatisant le déploiement de ces services avec Docker ?

CONTRAINTES

- MERN
- Architecture micro-services
- API Gateway
- Solution de conteneurisation (Docker)

GÉNÉRALISATION

- Sécurité
- Conteneurisation
- Routage
- Micro-services

LIVRABLES

- Workshop

PISTES DE SOLUTIONS

- Configurer Nginx ?
- Utiliser un service de gestion d'identité (Auth0, Keycloak) ?
- Utiliser un orchestrateur de conteneurs (Kubernetes/EKS) ?

PLAN D'ACTIONS

- Comprendre les mécanismes d'authentification basés sur les jetons JWT
- Modéliser la nouvelle architecture intégrant le service d'authentification indépendant et l'API Gateway
- Implémenter la génération et la validation des jetons JWT dans le service d'authentification
- Adapter le code front-end pour gérer les jetons JWT et les inclure dans les requêtes vers les micro-services
- Configurer Nginx pour le routage vers les micro-services et la validation des jetons JWT
- Conteneuriser les micro-services et le service d'authentification avec Docker
- Automatiser le déploiement avec un orchestrateur de conteneurs (Kubernetes/EKS)

DÉFINITION : MOTS CLÉS

Prototype MERN

Un **prototype MERN** est une première version fonctionnelle d'une application web bâtie sur la **pile MERN**, un ensemble de quatre technologies **tout-JavaScript** :

- **M – MongoDB** : base de données NoSQL orientée documents (persistance).
- **E – Express** : framework web minimaliste pour Node.js (routes et API REST).
- **R – React** : bibliothèque front-end pour l'interface utilisateur (côté client).
- **N – Node.js** : environnement d'exécution JavaScript côté serveur.

Son intérêt majeur : un **langage unique (JavaScript)** de bout en bout et un format de données homogène (**JSON/BSON**) du navigateur jusqu'à la base. Dans le contexte du projet, c'est précisément ce **prototype MERN en micro-services** que le développeur Yanis doit **sécuriser** (via des jetons JWT) et **déployer** (via Docker).

Docker (conteneurisation)

Docker est une plateforme de **conteneurisation**. Un **conteneur** est un paquet **léger et isolé** qui embarque une application **avec toutes ses dépendances** (bibliothèques, runtime, configuration), garantissant qu'elle s'exécute **de façon identique** partout (« *ça marche sur ma machine* » devient « *ça marche partout* »).

À la différence d'une **machine virtuelle (VM)** qui embarque un système d'exploitation complet, les conteneurs **partagent le noyau de l'OS hôte** : ils sont donc beaucoup plus **légers, rapides à démarrer et économes** en ressources. C'est la solution de conteneurisation imposée par les contraintes du projet pour **automatiser le déploiement** des services de Yanis.

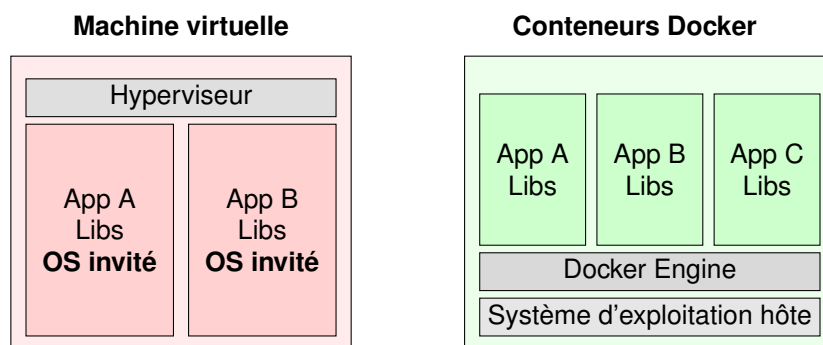


Figure : la VM virtualise tout un OS, le conteneur partage le noyau hôte (plus léger).

JWT (JSON Web Token)

Le **JWT** (*JSON Web Token*, norme **RFC 7519**) est un standard ouvert définissant un format **compact et autoportant** pour échanger de manière sécurisée des informations sous forme d'objet **JSON**. Un JWT se compose de **trois parties** encodées en Base64URL et séparées par des points :

- **En-tête** (*header*) : l'**algorithme** de signature (ex. HS256) et le type de jeton.
- **Charge utile** (*payload*) : les **revendications** (*claims*) — identité de l'utilisateur, rôle, date d'expiration (*exp*)...
- **Signature** : garantit l'**intégrité** et l'**authenticité** du jeton.

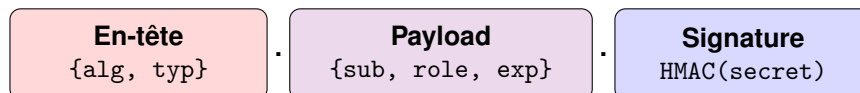


Figure : les trois parties d'un JWT — *xxxxx.yyyyy.zzzzz*.

Sa signature cryptographique permet au serveur de **vérifier la validité** du jeton **sans conserver d'état de session** (*stateless*) : idéal pour des micro-services indépendants. Exemple de *payload* :

```
{ "sub": "user_42", "name": "Yanis", "role": "admin", "exp": 1735689600 }
```

Jetons / Tokens

Un **jeton** (ou *token*) est une donnée numérique délivrée par un serveur d'authentification qui **matérialise et atteste des droits** d'un utilisateur au sein d'un système. Il remplace l'envoi répété des identifiants : une fois authentifié, l'utilisateur reçoit un jeton qu'il **joint à chacune de ses requêtes** (en-tête HTTP Authorization: Bearer <token>) pour prouver son identité. On distingue :

- l'**access token** (jeton d'accès) : **courte durée de vie**, présenté à chaque requête pour accéder aux ressources ;
- le **refresh token** (jeton de rafraîchissement) : **longue durée de vie**, qui permet d'obtenir un nouvel *access token* **sans se reconnecter**.

Dans le projet, ces jetons sont des **JWT** : c'est le mécanisme retenu pour sécuriser les échanges entre le front-end et les micro-services.

Authentification

L'**authentification** (*AuthN*) est le processus qui consiste à **vérifier l'identité** d'un utilisateur avant de lui accorder l'accès. Elle répond à la question « **Qui êtes-vous ?** » en confrontant une **preuve** (mot de passe, jeton, certificat, biométrie) à une référence connue du système. Elle se distingue de l'**autorisation** (« qu'avez-vous le droit de faire ? »), qui intervient **ensuite**. Dans une architecture à jetons, l'authentification aboutit à la **délivrance d'un JWT** :

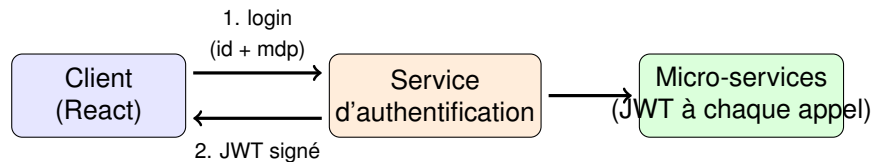


Figure : l'utilisateur s'authentifie une fois, reçoit un JWT, puis le présente à chaque micro-service. L'authentification peut être renforcée par **plusieurs facteurs (MFA)**. C'est l'objet de l'AAV « Réaliser un système d'authentification dans une API ».

API Gateway

Une **API Gateway** (passerelle d'API) est le **point d'entrée unique** d'une architecture micro-services. Tous les appels clients passent par elle, et elle se charge de les **router** vers le bon service. Elle centralise des préoccupations transverses qu'il serait coûteux de dupliquer dans chaque service :

- **routing** et agrégation des requêtes ;
- **authentification / autorisation** : **validation des jetons JWT** en amont des services ;
- **limitation de débit** (*rate limiting*), journalisation, mise en cache ;
- **terminaison TLS** et masquage de la structure interne.

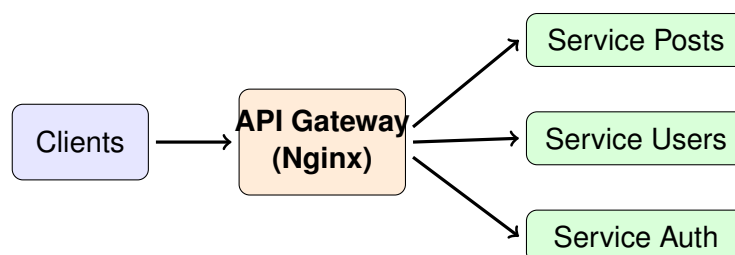


Figure : l'API Gateway (Nginx) route les requêtes et valide les JWT avant d'atteindre les micro-services.

Micro-services

L'**architecture en micro-services** (MSA) consiste à découper une application en un ensemble de **petits services autonomes**, chacun responsable d'une seule capacité métier (posts, utilisateurs, authentification...). Chaque service :

- est **développé, déployé et mis à l'échelle indépendamment** des autres ;
- possède idéalement sa **propre base de données** (*Database per Service*) ;
- communique via des **interfaces réseau légères**, le plus souvent des **API REST** en HTTP/JSON.

Avantages : scalabilité fine, résilience (une panne reste isolée), déploiements indépendants. **Inconvénients** : complexité accrue (réseau, supervision) et nécessité d'une **API Gateway** ainsi que d'une stratégie de **sécurité distribuée** — ce qui justifie précisément le recours aux **jetons JWT** dans ce projet, chaque service vérifiant le jeton de façon autonome.

Multi-conteneurs

Une application **multi-conteneurs** est une application dont les différents composants (chaque micro-service, sa base de données, la passerelle) s'exécutent dans des **conteneurs distincts mais coordonnés**. Chaque composant a son propre environnement isolé tout en communiquant via un **réseau partagé**. L'orchestration est assurée par **Docker Compose** (développement / déploiements simples) ou par un **orchestrateur** comme **Kubernetes** (production à grande échelle). Un fichier `docker-compose.yml` déclare l'ensemble des services :

```
services:
  gateway: { image: nginx, ports: ["80:80"] }      # passerelle
  auth:    { build: ./auth-service }              # délivre les JWT
  users:   { build: ./users-service }             # micro-service métier
  mongo:   { image: mongo }                       # base de données
```

Une seule commande (`docker compose up`) démarre alors **toute la pile** de Yanis : passerelle, service d'authentification, micro-services et base de données.

Autorisation (Contrôle d'accès)

L'**autorisation** (*AuthZ*), ou **contrôle d'accès**, détermine les **ressources et actions** auxquelles un utilisateur **déjà authentifié** a le droit d'accéder. Elle répond à la question « **Qu'avez-vous le droit de faire ?** » et intervient **toujours après** l'authentification. Le modèle le plus courant est le **RBAC** (*Role-Based Access Control*) : des **permissions** sont attribuées à des **rôles**, eux-mêmes assignés aux utilisateurs.

Rôle	Lire	Supprimer
visiteur	oui	non
membre	oui	ses propres posts
admin	oui	tous les posts

Dans une architecture à jetons, le **rôle** de l'utilisateur est encodé dans les **revendications** (*claims*) du **JWT** et **vérifié à chaque requête** par le micro-service concerné. C'est la réponse aux AAV « Mettre en place le contrôle d'accès dans une application backend » et « Proposer un contrôle d'accès pour une application front ».

ANALYSE ET RÉOLUTION : JWT DOCKER

Rappel de la problématique : « Comment sécuriser l'accès aux micro-services d'une application web MERN, en utilisant des jetons JWT pour l'authentification et le contrôle d'accès, tout en automatisant le déploiement de ces services avec Docker ? »

La réponse se construit en six étapes : **(1)** le choix du **mécanisme d'authentification** par jeton (service maison vs service tiers), **(2)** l'**architecture cible** (service d'authentification indépendant derrière une API Gateway), **(3)** la **génération et la validation des JWT** dans l'API, **(4)** le **contrôle d'accès back-end**, **(5)** le **contrôle d'accès front-end**, et **(6)** la **conteneurisation Docker** et l'automatisation du déploiement. Une synthèse argumentée clôt l'analyse.

1. MÉCANISME D'AUTHENTIFICATION : POURQUOI DES JETONS JWT

Dans une architecture **micro-services**, l'authentification par **session serveur** est mal adaptée : elle suppose un état partagé entre des services censés rester **indépendants et sans état**. La solution retenue est l'authentification par **jeton JWT** : après connexion, l'utilisateur reçoit un jeton **signé** qu'il présente à chaque requête, et que **chaque service vérifie seul** grâce à la signature, sans interroger une base de sessions centrale.

Deux approches sont possibles pour **émettre** ces jetons :

- **Service maison** : un micro-service d'authentification développé en Node.js (bibliothèque `jsonwebtoken`) qui signe et vérifie les JWT. Simple, totalement maîtrisé,

suffisant pour le périmètre du projet.

- **Service tiers d'identité (Auth0, Keycloak)** : un fournisseur externe qui délègue l'authentification via les protocoles standard **OAuth 2.0 / OpenID Connect**. L'application ne gère plus les mots de passe ; elle reçoit un JWT émis par le tiers. Avantages : **SSO**, MFA, gestion des comptes externalisée et conformité.

Décision : un **service d'authentification dédié** est créé (réponse à l'AAV « Réaliser un système d'authentification dans une API »), l'architecture restant **compatible** avec un futur basculement vers un **service tiers** (Keycloak), puisque dans les deux cas les services consommateurs ne font que **valider un JWT**.

2. ARCHITECTURE CIBLE : SERVICE D'AUTHENTIFICATION + API GATEWAY

La nouvelle architecture isole l'authentification dans un **micro-service indépendant** et place une **API Gateway (Nginx)** en **point d'entrée unique**. Le parcours d'une requête est le suivant :

1. le client s'authentifie auprès du **service d'authentification** et obtient un **JWT** ;
2. il joint ce jeton (`Authorization: Bearer <token>`) à chacun de ses appels ;
3. la **Gateway** reçoit la requête, **valide le JWT**, puis la **route** vers le micro-service métier visé ;
4. le micro-service, qui reçoit une requête déjà authentifiée, applique le **contrôle d'accès** selon le rôle.

Cette séparation respecte le principe **une responsabilité par service** et centralise la sécurité au bon endroit (la Gateway pour la **validation**, chaque service pour l'**autorisation** métier).

3. GÉNÉRATION ET VALIDATION DES JWT DANS L'API

Le service d'authentification expose une route de **connexion** qui vérifie les identifiants, puis **signe un JWT** contenant les *claims* utiles (identifiant et rôle), avec une **courte durée de vie** :

```
const jwt = require('jsonwebtoken');
const SECRET = process.env.JWT_SECRET; // secret en variable d'env.

// POST /api/auth/login -> authentifie puis délivre un JWT
async function login(req, res) {
  const user = await User.findOne({ email: req.body.email });
  const ok = user && await bcrypt.compare(req.body.password, user.hash);
  if (!ok) return res.status(401).json({ erreur: 'Identifiants invalides' });
```

```

const token = jwt.sign(
  { sub: user.id, role: user.role },           // claims (identité + rôle)
  SECRET,
  { expiresIn: '15m' }                       // jeton à courte durée de vie
);
res.json({ token });
}

```

Côté micro-services, un **middleware** extrait le jeton de l'en-tête Authorization et le **vérifie** (signature + expiration). En cas d'échec, l'accès est refusé (401) :

```

// Middleware : vérifie le JWT présent dans l'en-tête Authorization
function authentifier(req, res, next) {
  const header = req.headers.authorization || '';
  const token = header.startsWith('Bearer ') ? header.slice(7) : null;
  if (!token) return res.status(401).json({ erreur: 'Jeton manquant' });
  try {
    req.utilisateur = jwt.verify(token, SECRET); // payload vérifié et décodé
    next();
  } catch {
    res.status(401).json({ erreur: 'Jeton invalide ou expiré' });
  }
}

```

C'est la réponse aux AAV « Réaliser un système d'authentification dans une API » et « Gérer la sécurité de l'application à l'aide de jetons » : le jeton, **signé et daté**, est validé de manière **autonome** par chaque service.

4. CONTRÔLE D'ACCÈS CÔTÉ BACK-END (RBAC)

Une fois l'utilisateur **authentifié**, le service applique l'**autorisation** en lisant le **rôle** contenu dans les *claims* du JWT. Un second middleware, paramétré par les rôles autorisés, implémente le **RBAC** :

```
// Contrôle d'accès basé sur les rôles (RBAC)
function autoriser(...roles) {
  return (req, res, next) => {
    if (!roles.includes(req.utilisateur.role))
      return res.status(403).json({ erreur: 'Accès interdit' }); // 403
    next();
  };
}

// Exemple : seul un admin peut supprimer un post
router.delete('/api/posts/:id', authentifier, autoriser('admin'), supprimerPost);
```

La chaîne authentifier → autoriser → contrôleur sépare clairement les deux responsabilités : **qui es-tu** (401 si le jeton est invalide) puis **as-tu le droit** (403 si le rôle est insuffisant). C'est la réponse à l'AAV « Mettre en place le contrôle d'accès dans une application backend ».

5. CONTRÔLE D'ACCÈS CÔTÉ FRONT-END

Côté React, deux mécanismes sont mis en place. D'abord un **intercepteur Axios** qui **joint automatiquement le JWT** à chaque requête sortante :

```
// Intercepteur Axios : ajoute le JWT à chaque requête
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

Ensuite des **routes protégées** qui masquent ou redirigent selon l'état d'authentification et le rôle :

```
// Route protégée : redirige si non authentifié ou rôle insuffisant
function RouteProtegee({ children, role }) {
  const user = useAuth(); // décode le JWT stocké
  if (!user) return <Navigate to="/login" />;
  if (role && user.role !== role) return <Navigate to="/403" />;
  return children;
}
```

Point de sécurité essentiel : le contrôle d'accès front-end relève uniquement de l'**expérience utilisateur** (ne pas afficher de boutons inutiles, rediriger proprement). Il

est **contournable** et ne constitue **jamais** une protection : la sécurité réelle est **toujours imposée côté back-end**. C'est la réponse à l'AAV « Proposer un contrôle d'accès pour une application front ».

6. CONTENEURISATION DOCKER ET AUTOMATISATION DU DÉPLOIEMENT

Chaque brique (service d'authentification, micro-services, Gateway, base) est **conteneurisée** via un Dockerfile dédié, garantissant une exécution **identique** du poste de Yanis à la production :

```
FROM node:20-alpine          # image légère
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev        # dépendances de production uniquement
COPY . .
EXPOSE 4000
CMD ["node", "server.js"]    # point d'entrée du service
```

La **Gateway Nginx** route le trafic et **délègue la validation du jeton** au service d'authentification (sous-requête `auth_request`) avant de transmettre aux services métier :

```
upstream auth { server auth:4000; }
upstream users { server users:5000; }

server {
    listen 80;
    location /api/auth/ { proxy_pass http://auth; } # login / refresh (public)
    location /api/users/ {
        auth_request /verify; # valide le JWT en amont
        proxy_pass http://users;
    }
    location = /verify { internal; proxy_pass http://auth/verify; }
}
```

Enfin, l'ensemble est orchestré en **multi-conteneurs** par **Docker Compose** (cf. `docker-compose.yml` en partie « Mots clés ») : une seule commande `docker compose up` démarre toute la pile. Pour la **production à grande échelle**, ce même socle conteneurisé se déploie sur un **orchestrateur** (Kubernetes / AWS EKS), qui apporte **scalabilité**, **auto-réparation** et déploiements progressifs sans interruption — répondant à l'objectif d'**automatisation du déploiement**.

7. ARCHITECTURE RETENUE (SCHÉMA ARGUMENTÉ)

L'architecture cible combine tous ces éléments : le client React obtient un JWT auprès du **service d'authentification**, puis tous ses appels transitent par la **Gateway Nginx** qui **valide le jeton** et **route** vers des **micro-services conteneurisés**, chacun appliquant son **contrôle d'accès** et disposant de sa **propre base MongoDB**, le tout déployé en **conteneurs Docker**.

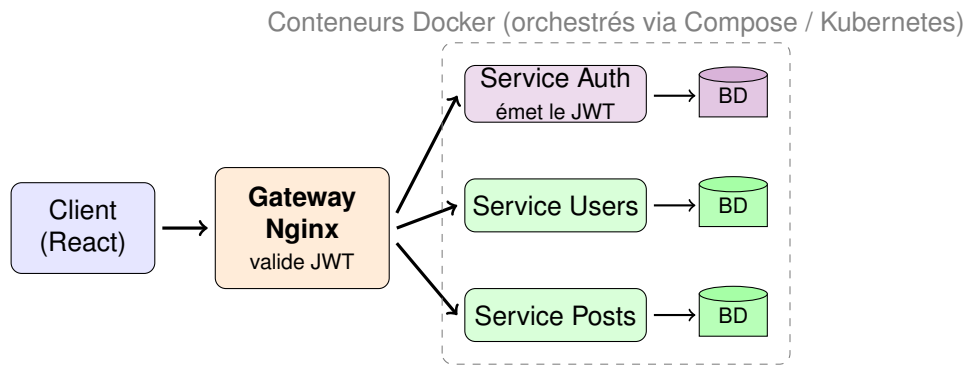


Figure : architecture retenue — service d'authentification émettant les JWT, Gateway Nginx validant et routant, micro-services conteneurisés avec base par service.

Justification : chaque exigence de la problématique est satisfaite — **authentification** par un service dédié, **sécurité par jetons JWT** signés et validés sans état, **contrôle d'accès** RBAC back-end (et UX front-end), point d'entrée unique **Gateway Nginx**, et **déploiement automatisé** grâce à Docker et à l'orchestration.

CONCLUSION

En réponse à la problématique, sécuriser l'accès aux micro-services d'une application MERN ne tient pas à une mesure isolée mais à une **chaîne de décisions cohérentes**. L'authentification est confiée à un **service dédié** qui délivre des **JWT** signés et à courte durée de vie ; ce choix *stateless* est parfaitement adapté aux micro-services, qui **valident chacun le jeton de façon autonome** sans état partagé, tout en laissant la porte ouverte à un **service tiers** (Keycloak) si le besoin grandit.

La **sécurité par jetons** se décline ensuite en deux temps : l'**authentification** (401 si le jeton est absent ou invalide) puis l'**autorisation RBAC** fondée sur le rôle inscrit dans les *claims* (403 si le rôle est insuffisant). Côté front-end, les routes protégées et l'intercepteur Axios améliorent l'expérience, mais la **protection réelle reste imposée côté back-end**. Une **API Gateway Nginx** centralise la validation des jetons et le routage en point d'entrée unique.

Enfin, la **conteneurisation Docker** et l'orchestration **multi-conteneurs** (Docker Compose en développement, Kubernetes/EKS en production) **automatisent le déploiement** et assurent une exécution identique de bout en bout. L'ensemble constitue un **modèle reproductible** : la démarche validée ici pourra être étendue, service après service, à toute l'application de l'entreprise.

CAPITALISATION

AAV	Commentaire	Statut
Expliquer le fonctionnement d'un service tiers d'authentification	Service maison (jsonwebtoken) comparé à un service tiers (Auth0/Keycloak, OAuth 2.0/OIDC) : rôle du serveur d'autorisation et émission déléguée du JWT.	Acquis
Gérer la sécurité de l'application à l'aide de jetons	JWT signés à courte durée de vie (+ <i>refresh</i>), transmis en Bearer et validés sans état par chaque micro-service ; structure en-tête/payload/signature détaillée.	Acquis
Mettre en place le contrôle d'accès dans une application backend	Middleware RBAC vérifiant le rôle inscrit dans les <i>claims</i> du JWT (chaîne authentifier → autoriser, 401/403), illustré par du code Express.	Acquis
Proposer un contrôle d'accès pour une application front	Routes protégées React et intercepteur Axios joignant le JWT ; rappel que le contrôle front est UX et que la protection réelle reste côté back-end.	Acquis
Réaliser un système d'authentification dans une API	Service d'authentification Express délivrant les JWT (route login, signature, expiration) et middleware de validation, illustrés par du code commenté.	Acquis

RESSOURCES

- **JWT (introduction & débogueur)** : <https://jwt.io/introduction>
- **RFC 7519 – JSON Web Token** : <https://datatracker.ietf.org/doc/html/rfc7519>
- **Bibliothèque jsonwebtoken (Node.js)** : <https://github.com/auth0/node-jwebtoken>
- **Keycloak (service d'identité)** : <https://www.keycloak.org/documentation>
- **OAuth 2.0 / OpenID Connect** : <https://openid.net/developers/how-connect-works/>
- **Nginx – module auth_request** : https://nginx.org/en/docs/http/nginx_http_auth_request_module.html
- **Documentation Docker & Compose** : <https://docs.docker.com/>
- **Amazon EKS (Kubernetes managé)** : <https://docs.aws.amazon.com/eks/>
- **OWASP – bonnes pratiques d'authentification** : https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html